Formal Verification meets AI

Using Lean as a Practical Bridge

Aziz Najjar

Formal Semantics, Spring 2026
Instructor: Dániel Horpácsi
ELTE Faculty of Informatics

Abstract

Core idea

Formal methods from this course apply directly to verifying AI systems. Lean 4 provides a practical bridge between theory and practice.

Roadmap:

- An AI failure that testing could not prevent
- Testing vs. formal proof — the fundamental gap
- Three verified Lean 4 properties of neural network components
- How each example connects to course concepts
- Where the field is heading: LeanDojo, AlphaProof, DeepSeek-Prover-V2

A Note on Tools

Retrieval (Claude + RAG)

Lecture PDFs 1, 3, 4 uploaded as context. Used to ground references to:

- compositionality
- static semantics
- derivation sequences

Lean 4 proofs

- Inspired by *Theorem Proving in Lean 4* (Avigad et al.)
- All proofs written and verified manually in VS Code

Next: what retrieval over lecture PDFs looks like in practice.

Querying Lecture Material

Claude generated these lecture summaries by querying Lectures 1, 3, and 4 as project knowledge — each diagram is grounded in the actual PDF content, not general knowledge.

Why Testing Is Not Enough

Dijkstra (1969)

"Testing shows the presence of bugs, not their absence."

Amazon Rekognition (2018)

28 US Congress members misidentified as criminals after extensive testing — the model was evaluated on the wrong distribution of inputs.

Testing

- Finite set of inputs
- One missed case can fail

Formal proof

- Guaranteed for *all* $x \in \mathbb{R}$
- No exceptions possible

aclu.org/news/privacy-technology/amazons-face-recognition-falsely-matched-28

From Semantics to Lean

Course concept	Lean counterpart
Inference rules (Lec. 3, 4)	Tactics apply rules step-by-step — a derivation tree
Compositionality (Lec. 3)	Proofs compose from sub-proofs, like $[\![e_1+e_2]\!]s$
Static semantics (Lec. 1)	Dependent types enforce constraints at compile time
States: $\mathrm{Var} \to \mathbb{Z}$ (Lec. 3)	Lean's local context plays the role of the state function

Key idea: course definitions become checkable proof obligations. Lec. 3 writes $\mathcal{A}[\![a_1+a_2]\!]s = \mathcal{A}[\![a_1]\!]s + \mathcal{A}[\![a_2]\!]s$ on paper; Lean enforces it.

Example 1 / 3 ReLU (Rectified Linear Unit) is Non-Negative

relu.lean

```
noncomputable def relu (x : R) : R :=
  max 0 x

theorem relu_nonneg (x : R) :
    relu x >= 0 := by
  unfold relu        -- replace name with its body
  exact le_max_left 0 x -- close goal: 0 <= max 0 x
```

Why it matters

ReLU is the standard neural network activation function. This proof holds for *all* $x \in \mathbb{R}$ — no test suite can do that.

`unfold relu` — expands the definition: goal becomes $0 \leq \max\ 0\ x$

`exact le_max_left 0 x` — applies a Mathlib lemma to close the goal

Course link (Lec. 3): the theorem holds for all `x : R` — just as $\mathcal{A}[\![a]\!]s$ is defined for all states s. One proof, infinite coverage.

Example 2 / 3 Vector Dimension Safety

vecadd.lean

```
1  -- n is part of the TYPE, not a runtime variable
2  def vecAdd {n : N} (a b : Fin n -> R)
3      : Fin n -> R :=
4    fun i => a i + b i   -- add element by element
5
6  def v3 : Fin 3 -> R := ![1, 2, 3]  -- length 3
7  def v2 : Fin 2 -> R := ![4, 5]     -- length 2
8
9  #check vecAdd v3 v3  -- OK:  Fin 3 -> R
10 -- #check vecAdd v3 v2
11 -- ^ TYPE ERROR: Fin 3 /= Fin 2 (compile time)
```

Why it matters

Shape mismatch is the most common runtime crash in deep learning frameworks such as PyTorch and TensorFlow. Lean catches it *before* the program runs — at zero runtime cost.

No theorem to prove — the type system does the work

`{n : N}` — *n* is implicit: Lean infers the size from the input

`Fin n -> R` — a vector of exactly *n* reals; length lives in the type

*Course link (Lec. 1): static semantics enforces constraints a grammar cannot express. Here the constraint is: both vectors must have the same length **n**.*

Violated at compile time, not at runtime.

Example 3 / 3 MSE (Mean Squared Error) Loss is Non-Negative

mse.lean

```
-- ys = true labels, ys' = predictions
-- goal: sum of squared differences >= 0
theorem mse_nonneg {n : N}
    (ys ys' : Fin n -> R) :
    0 <= sum i, (ys i - ys' i) ^ 2 := by
  apply Finset.sum_nonneg
  -- reduce to: each term >= 0
  intro i _      -- pick one arbitrary index i
  exact sq_nonneg _
  -- any square is >= 0, goal closed
```

Why it matters

MSE is the most common regression loss function. A verified lower bound of 0 eliminates a class of numerical instability bugs that can silently corrupt training.

`apply sum_nonneg` — reduces the sum to proving each term ≥ 0

`intro i _` — pick an arbitrary index; discard membership proof

`exact sq_nonneg` — any real square is ≥ 0; goal closed

*Course link (Lec. 3): proof composes from parts, like $\mathcal{A}[\![a_1 + a_2]\!]s = \mathcal{A}[\![a_1]\!]s + \mathcal{A}[\![a_2]\!]s$. **sq_nonneg** handles one term; **sum_nonneg** lifts it to the whole sum.*

Try It Yourself

GitHub repository

github.com/Njoura7/formal-verification-ai

What is there

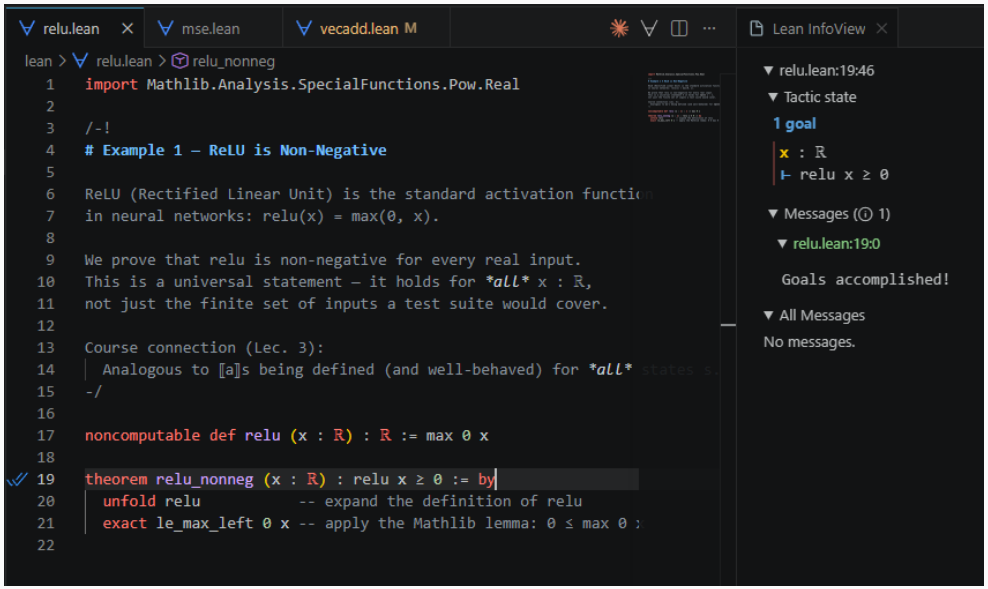
- *lean/relu.lean*
- *lean/vecadd.lean*
- *lean/mse.lean*
- *README.md* with setup steps

To run in VS Code

1. Install *leanprover.lean4* extension
2. *lake exe cache get*
3. Open any *.lean* file
4. See Goals accomplished!

All three proofs compile against Mathlib — reproducible from a fresh install.

Lean in VS Code



What you see

Left — proof file with tactics

Right — Lean Infoview:

- Tactic state shows the open goal
- $\mathit{relu}\ x \geq 0$ waiting to be closed
- Goals accomplished! once all tactics apply

When AI Starts Proving Things

LeanDojo

ML research conference (NeurIPS), 2023 — open source

Turns Lean's proof state into a training environment. A language model learns to pick the next tactic — one step at a time, like building a derivation tree.

AlphaProof

Google DeepMind, 2024 — closed research

Translated IMO competition problems into Lean, then trained on proof search. Solved 4 of 6 problems — equivalent to a silver medal.

DeepSeek Prover V2

DeepSeek AI, 2025 — open source, preprint

Decomposes hard theorems into sub-goals before proving each one separately — the same compositional structure as denotational semantics (Lec. 3).

None of these systems reason in natural language. They work inside Lean, where every step is type-checked.

The formal language is what makes the result trustworthy.

Conclusion

- Testing is **incomplete** — a proof is not
- Lean tactics *are* derivation sequences from the course
- ReLU, MSE, and shape safety are **formally provable** today
- AI systems are beginning to find those proofs themselves

Takeaway

AI systems can move from *working* to **provably correct**.

References i

- Yang, K. et al. *LeanDojo: Theorem Proving with Retrieval-Augmented Language Models*. NeurIPS 2023. **arXiv:2306.15626**
- DeepMind. *AI achieves silver-medal standard solving IMO problems*. Blog post, July 2024. **deepmind.google**
- Ying, H. et al. *DeepSeek-Prover-V2*. **arXiv:2504.21801**, 2025.
- ACLU. *Amazon's Face Recognition Falsely Matched 28 Members of Congress*. 2018. **aclu.org**
- Horpácsi, D. *Formal Semantics* — Lecture Notes, Lec. 1, 3, 4. ELTE, Spring 2026.
- Avigad, J. et al. *Theorem Proving in Lean 4*. **leanprover.github.io**

Questions?